

REFERENCE MATERIALS

SymPy Reference Sheets

Sympy is a useful Python library for doing computation with symbolic math. This reference covers a variety of useful and important SymPy functions that may be used in Calculus I & II classes at Texas A&M University (MATH 151, MATH 152).

In addition, you can learn more about SymPy by exploring its official documentation page [here](#). Another math library often used with SymPy is [matplotlib](#). Note that SymPy does have plotting functionality as well.

How to navigate this resource

The first few pages provide a quick introduction to basic SymPy workflow. If you only want to review specific functions or syntax, you may skip to the last few slides titled "Appendix".





SymPy

Setup

SymPy is an external Python library and *must* be imported before use. To do this, use the line of code below

```
import sympy
```

Additionally, you may add `as sp` at the end to use a shortened form

```
import sympy as sp
```

For sake of simplicity and consistency, we will be referring to SymPy as `sp` in these reference sheets. This is also the most commonly accepted convention.



SymPy

General Information

Variables (or symbols) can be represented in SymPy using `sp.symbols`

example: `x = sp.symbols("x")` `y = sp.symbols("y")`

This allows us to use variables in expression and have Python treat them similarly to mathematical variables. This means we can do entire computations in purely variables without the requirement for numerical output.

A major use case for this is defining functions. The syntax is as given

`function_name = expression` example: `f_x = 4*x + 5*y**2`

Notice that we did *not* define any numerical values for `x` and `y`. They are simply treated as variables in place of actual numbers, which will be useful for doing substitution and symbolic computation, which we will see later.



Symbolic Computation

Because we can represent whole functions as just expressions, we can also **substitute** values into them.

```
example: f_x = x**2 + x - 12
```

```
output = f_x.subs(x, 3)
```

The syntax for substitution is simply done by calling the `.subs` method on your function.

After calling `print(output)`, the program prints `0`

Functions can also be substituted into each other. For example, consider a composite function `f(g(x))`.

This can be represented by the following syntax

```
# define a function g(x)
```

```
g_x = 4 + 3*x
```

```
f_gx = g_x.subs(f_x) # now f_gx is equivalent to 4 + 3 * (x ** 2 + x - 12)
```



SymPy

Solving for unknowns

Sympy provides us a very useful method called `sp.solve()`. This method is used to solve for the roots of a function. Alternatively, it can also be used to solve systems of equations.

Let's see both cases. Consider a function $f(x)$ defined as `f_x = x**2 + x - 12`.

To use `sp.solve()`, the syntax is `sp.solve(your_function, variable)` where the variable is the one we are trying to solve for.

example: `solutions = sp.solve(f_x, x)`

SymPy



Solving for unknowns (pt. 2)

Now `sp.solve()` typically returns a list of solutions, even if there is only one solution. In our case, it would return `[-4, 3]`.

You may do whatever you wish with the output. Printing it directly is an option. You may also consider “unpacking” the list if you know how many solutions or the specific solution you want

Example: `sol1, sol2 = sp.solve(f_x, x)`

This works since we know that solving the quadratic equation above will product two solutions. You may also extract these values by indexing the list that you get back from `sp.solve()`.



Limits and Differentiation

Two essential tasks you'll be using SymPy a lot for are limits and differentiation, especially if you are taking Calculus I (MATH 151). It is good to get familiar with how to do these.

For limits, the syntax is as follows

```
lim = sp.limit(function, variable, value)
```

where value is the value the limit approaches

```
example: lim = sp.limit(sp.sin(x) / x, x, 0)
```

Derivatives are fairly intuitive as well. The syntax for differentiation is

```
f_px = sp.diff(function, variable)
```

```
example: f_px = sp.diff(sp.sin(x), x)
```

Additionally, we may specify a third parameter to indicate the order (e.g first order, second order)

```
example: f_px = sp.diff(sp.sin(x), x, 2)
```

This obtains the second derivative of `sin(x)`

SymPy

Integration



If you plan on taking Calculus II (MATH 152), you will have to perform a lot of integration. Integrals can be split into two categories: indefinite and definite integrals

We will look at both cases on this slide

Indefinite integrals

Given f_x , we can take its indefinite integral using

```
sp.integrate(f_x, x)
```

Note: SymPy does omit the $+C$

Definite Integrals

The only difference here is we will provide a range

```
sp.integrate(f_x, (x, lower, upper))
```


Appendix



Common Methods

Variables: `sp.symbols(var_name)`
Derivatives: `sp.diff(expr, var_name)`
Limits: `sp.limit(expr, var_name, value)`

Good to Know

Using infinity: `sp.oo` or `-sp.oo`
Exponential/Logs: `sp.exp()` # base e
`sp.log()` # ln()
Trig Functions: `sp.sin(x)`, `sp.cos(x)`, `sp.tan(x)`

Other Useful Methods

Solving: `sp.solve(expr, var_name)`
Simplify: `sp.simplify(expr)`
Expand: `sp.expand(expr)`

Equations

Form: `sp.Eq(left, right)`
N-solve: `sp.nsolve(expr, var, guess)`

Note: `sp.nsolve()` should only be used when an equation does not have a nice algebraic solution (e.g `x = cos(x)`)